

FINORA

Premium Expense Tracker Dashboard

Full-Stack MERN Development Internship Project

INTERNSHIP SUBMISSION REPORT

Organization: Codtech IT Solutions Private Limited

Intern ID: CITS1993

Full Name: Naguru Suhas

Duration: 8 Weeks

Submission Date: June 18, 2026

Table of Contents

1. Internship Details
2. Project Abstract
3. Project Objectives
4. Problem Statement
5. Solution Overview
6. Technology Stack
7. System Architecture
8. Folder Structure
9. Database Design
10. API Design
11. Authentication Flow
12. Core Features
13. Analytics Features
14. Security Features
15. Application Screenshots
16. Output Verification Images
17. Deployment Architecture
18. QA Testing Process
19. Challenges Faced
20. Future Enhancements
21. Conclusion

1. Internship Details

This project has been developed as the primary Capstone deliverable during the 8-Week MERN Stack Engineering Internship at Codtech IT Solutions Private Limited. The internship role involved building SaaS-grade full-stack features, setting up relational and non-relational database structures, configuring user authentication protocols, designing interactive visualization charts, and packaging the workspace for final audit review.

Intern Details

- **Intern ID:** CITS1993
- **Full Name:** Naguru Suhas
- **Duration:** 8 Weeks
- **Track:** Full-Stack Web Development
- **Project Title:** Finora – Expense Tracker Dashboard

2. Project Abstract

Finora is a premium full-stack wealth monitoring and expense auditing platform engineered with the MERN stack. Designed with SaaS-grade dark/light visual themes, the application targets cash management inefficiencies by providing real-time aggregation of balances, visual expenditure breakouts, monthly budget gauges, and savings goal trackers.

By integrating secure cookie-based JWT authentications alongside Google OAuth interfaces, users receive a protected profile space. With data exports configured for CSV sheets and print-ready PDF files, Finora offers comprehensive financial tracking capability suited for modern portfolio-ready submissions.

3. Project Objectives

- Provide a responsive, high-performance financial control interface.
- Build secure credentials logging utilizing bcrypt hashes and JWT session tokens.
- Offer intelligent analytics visualizations utilizing SVG charting engines.
- Configure monthly budgeting systems featuring alerts and status monitors.
- Deliver automated data serialization allowing users to export transaction ledgers as CSV and PDF files.

4. Problem Statement

Traditional bookkeeping remains complex, tedious, and prone to user errors. Many personal finance trackers require manual ledger compilation, fail to display graphical comparisons of savings versus expenses, and lack warning notifications when budgets are exceeded.

Additionally, developers and researchers often fail to find open-source finance dashboards that combine clean codebase structures, safe local database fallback mock engines, and production-ready credentials encryption with sleek, premium UI/UX aesthetics.

5. Solution Overview

Finora solves these challenges by providing a visual, secure workspace:

- **Smart Dashboards:** Live displays of Net Balance, Total Income, Expenses, Savings rate, and AI financial recommendations.
- **Interactive Recharts:** SVG line curves, bar comparisons, and category distribution charts updated dynamically.
- **Safe Fallbacks:** Fully-seeded in-memory demoStore backup system that runs automatically if local MongoDB connections are unavailable.
- **Secure API Layer:** Hardened security headers, CORS origins checks, and payload rate limits protecting against scraping or injections.

6. Technology Stack

Frontend Web Client

- **React (v18.3.1) & Vite:** SPA core framework and high-speed asset bundler.
- **Tailwind CSS (v3.4):** Utility-first layout styles with dark-theme toggles.
- **Framer Motion (v11.18):** Smooth transitions, page entry animations, and hover micro-triggers.
- **Recharts (v2.15):** Responsive charting components.
- **jspdf & jspdf-autotable:** Local PDF builder.

Backend API Service

- **Node.js & Express.js (v4.21):** REST endpoint hosting and controller structures.
- **MongoDB & Mongoose (v8.9):** Document database hosting and schema mapper.
- **JWT & Bcryptjs:** Authorization sessions and passwords encryption.
- **Helmet & Rate Limiters:** Endpoint security headers and spam prevention.

7. System Architecture

Finora uses a clean decoupled model. The React frontend operates independently, communicating with the Node.js/Express server using standard REST API queries over secure HTTPS. Dynamic state contexts manage session status and UI themes local preferences.

8. Folder Structure

```
backend/
├─ config/           # Database connection setup
├─ controllers/      # Authentication, transaction, and budget controllers
├─ middleware/       # Authorization check, security headers, validation
├─ models/           # Users, Transactions, Budgets, SavingsGoals models
├─ routes/           # Express route bindings
├─ utils/            # Async handler, JWT helpers, demoStore fallback
└─ server.js         # Entry server script
frontend/
├─ src/
│  ├─ api/           # Axios instance and HTTP configurations
│  ├─ components/    # Charts, layout modules, custom Buttons/Modals
│  ├─ context/       # Auth and Theme provider states
│  ├─ pages/         # Dashboard, Transactions, Analytics, Profile pages
│  └─ main.jsx       # App mount point
screenshots/         # App screenshot assets
output-images/       # VERIFIED success flow captures
documentation/       # Compiled reports
```

9. Database Design

Finora uses four primary schema collections. For details on fields, indexes, and relations, please refer to `DATABASE_DESIGN.md`. All collections enforce reference checks via user `ObjectIds`.

10. API Design

The backend routes map REST verbs to controller modules. All routes under `/api/transactions`, `/api/budgets`, `/api/analytics`, `/api/profile`, and `/api/savings-goals` are JWT-protected. Full route parameters are documented in `API_DOCUMENTATION.md`.

11. Authentication Flow

Traditional login encrypts input passwords and checks matching records via `bcrypt`. Google OAuth passes credentials to Google authentication client, creating a verified user index. Successful logins return a signed JWT token stored on client state.

12. Core Features

- **Register/Login/Reset:** Secure account flows with validation rules.
- **Transaction Ledger:** Create, view, edit, and delete transactions.
- **Export Tools:** Download current filters state as CSV spreadsheets or printable PDF tables.

13. Analytics Features

- **Income vs Expenses:** Dual bars tracking historical earnings against bills.
- **Category Pie Chart:** Breakdown of spending distributions.
- **Savings Curves:** Trend tracker modeling net monthly savings over time.

14. Security Features

- **Helmet Headers:** Hardened headers to block clickjacking and malicious scripts.
- **Rate Limiter:** Restricts authentication traffic to 30 requests per 15 minutes window.

15. Application Screenshots

The following images represent the actual Finora application layout captured during live execution under demo mode.

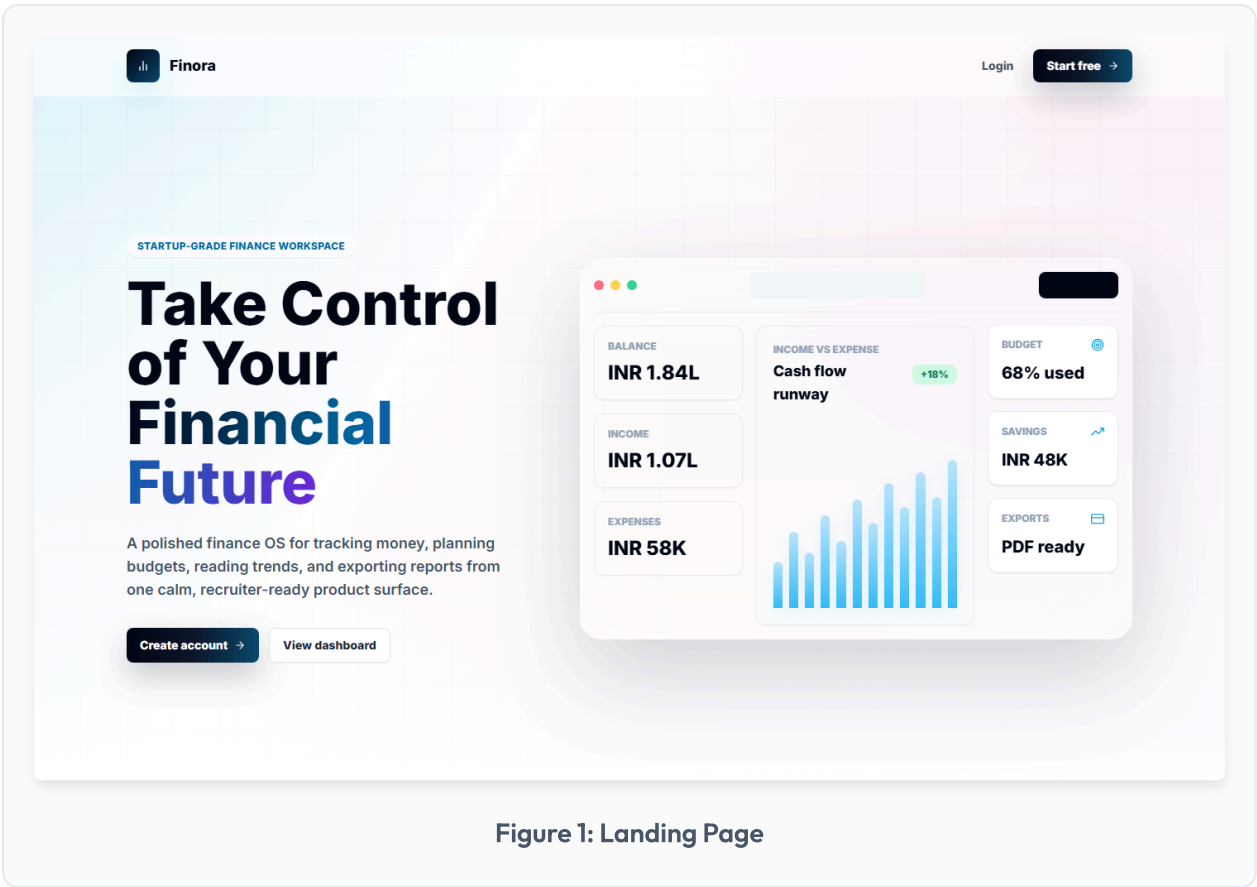


Figure 1: Landing Page

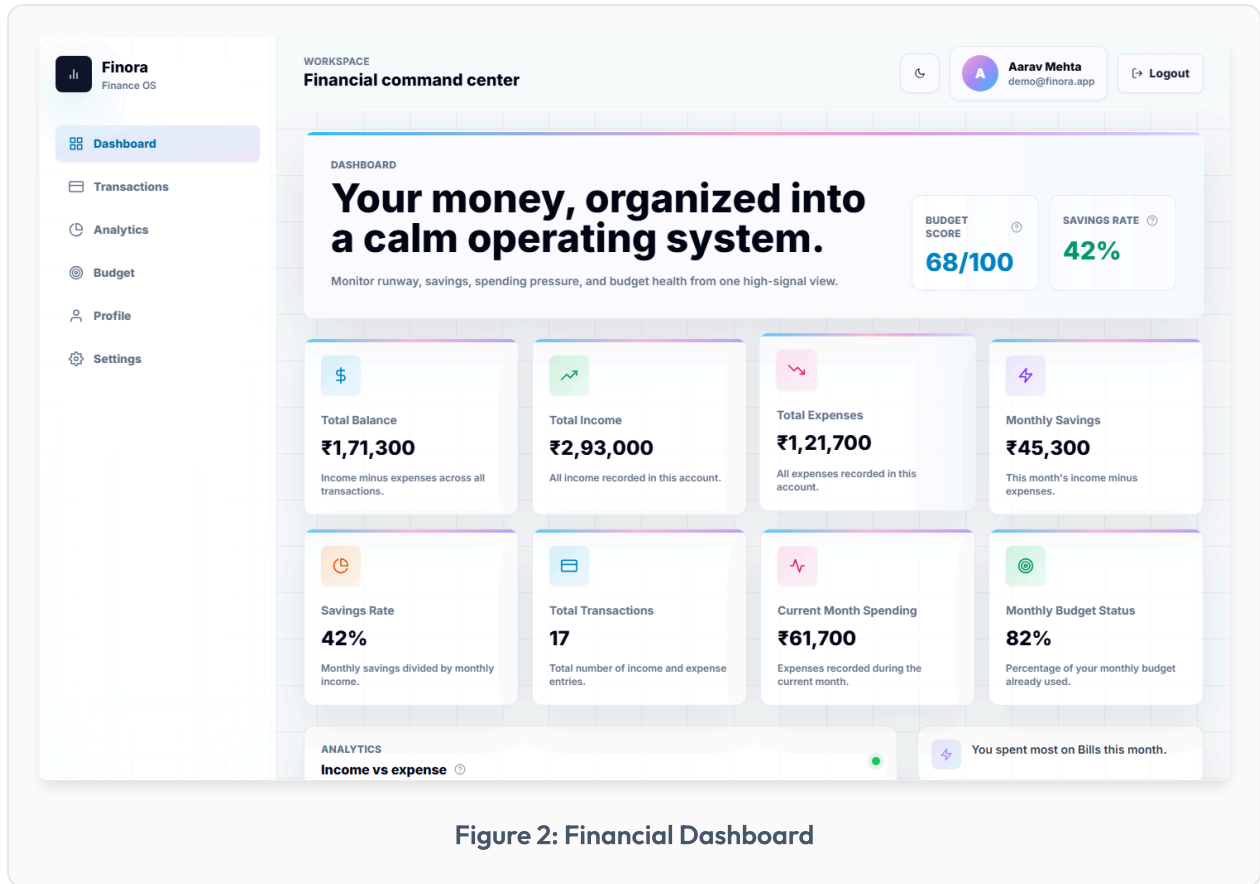
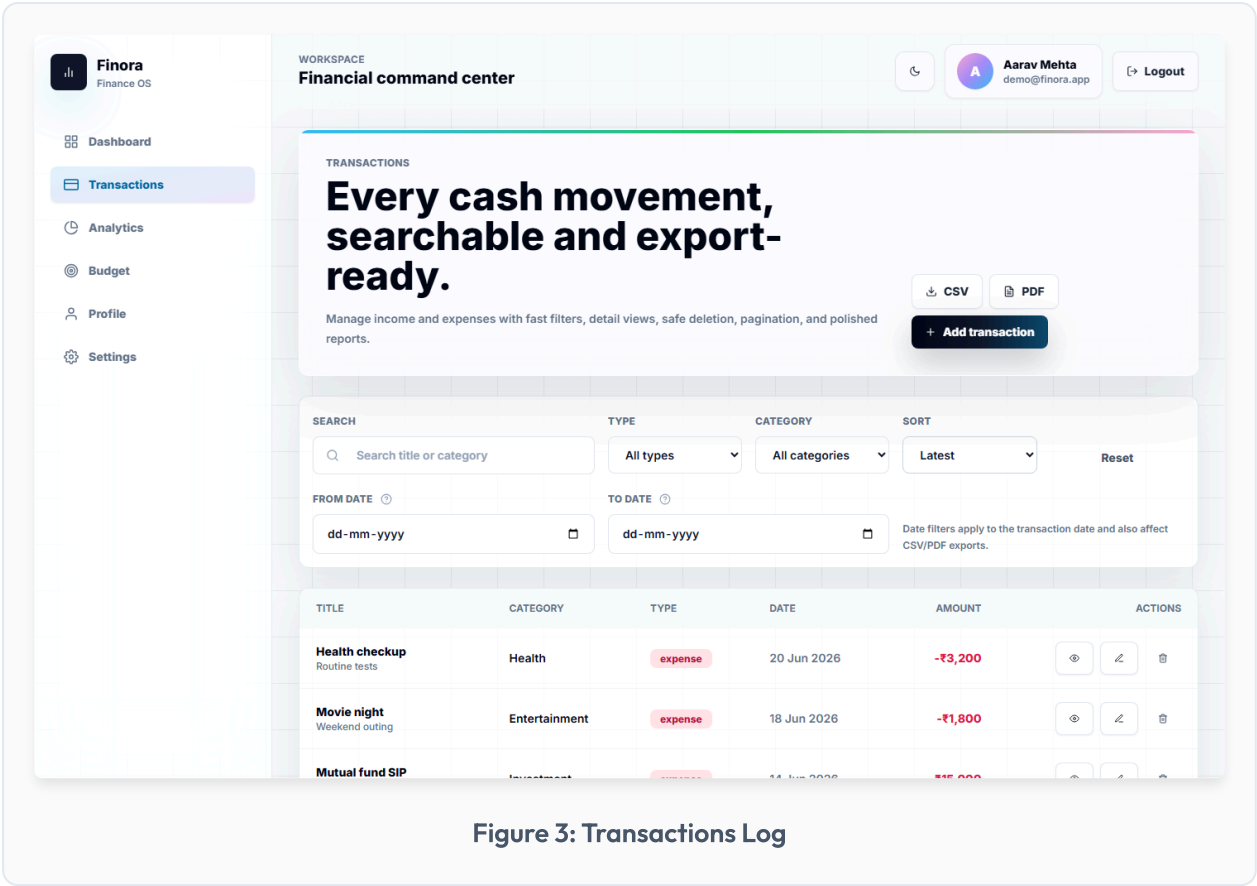


Figure 2: Financial Dashboard



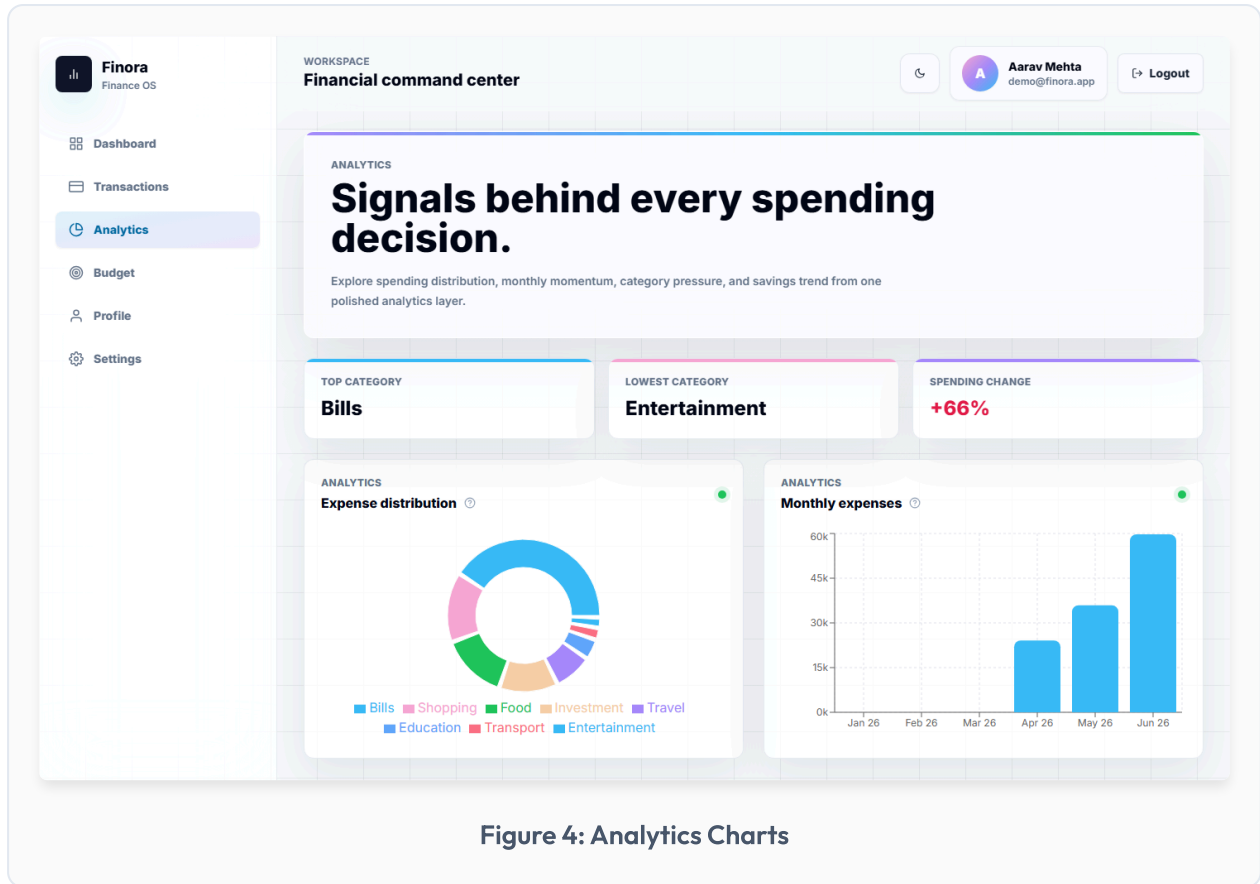
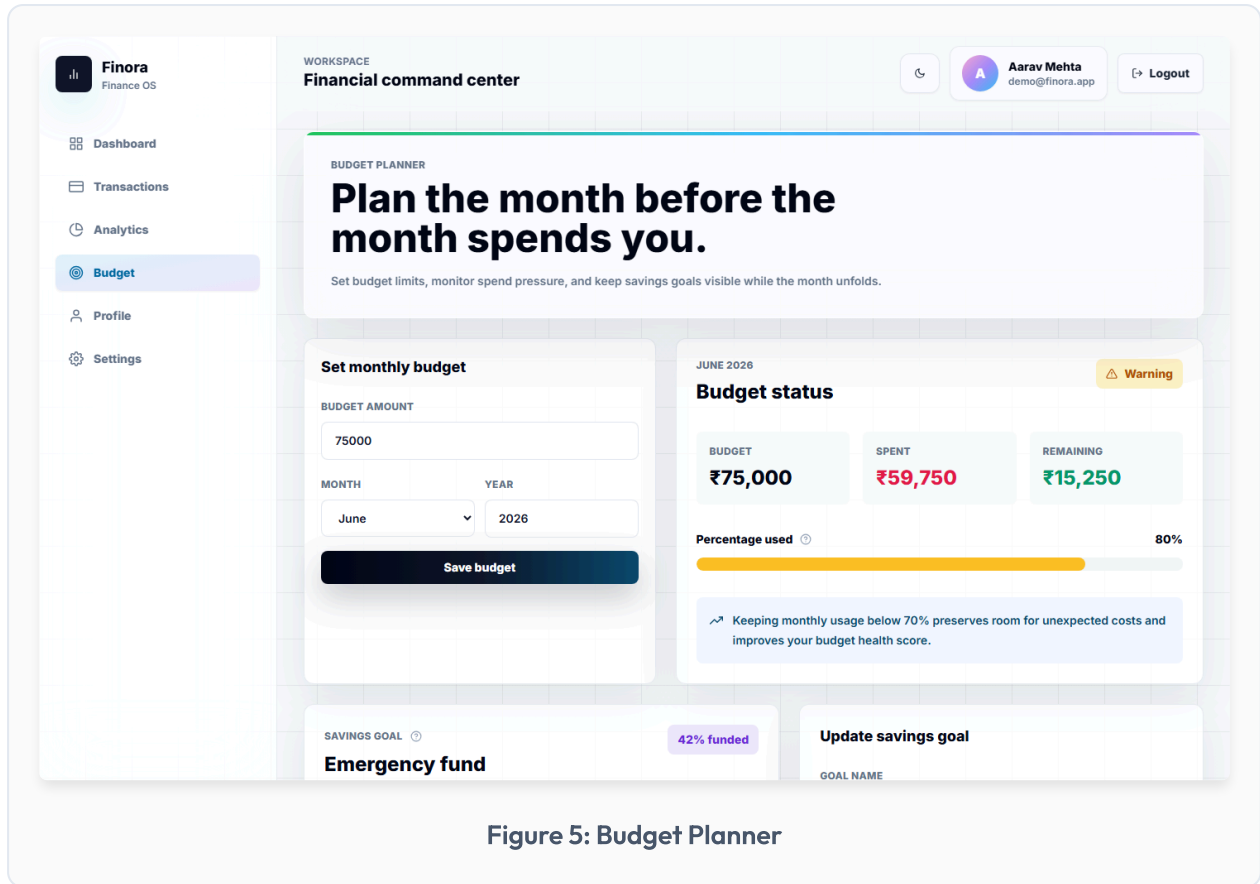


Figure 4: Analytics Charts



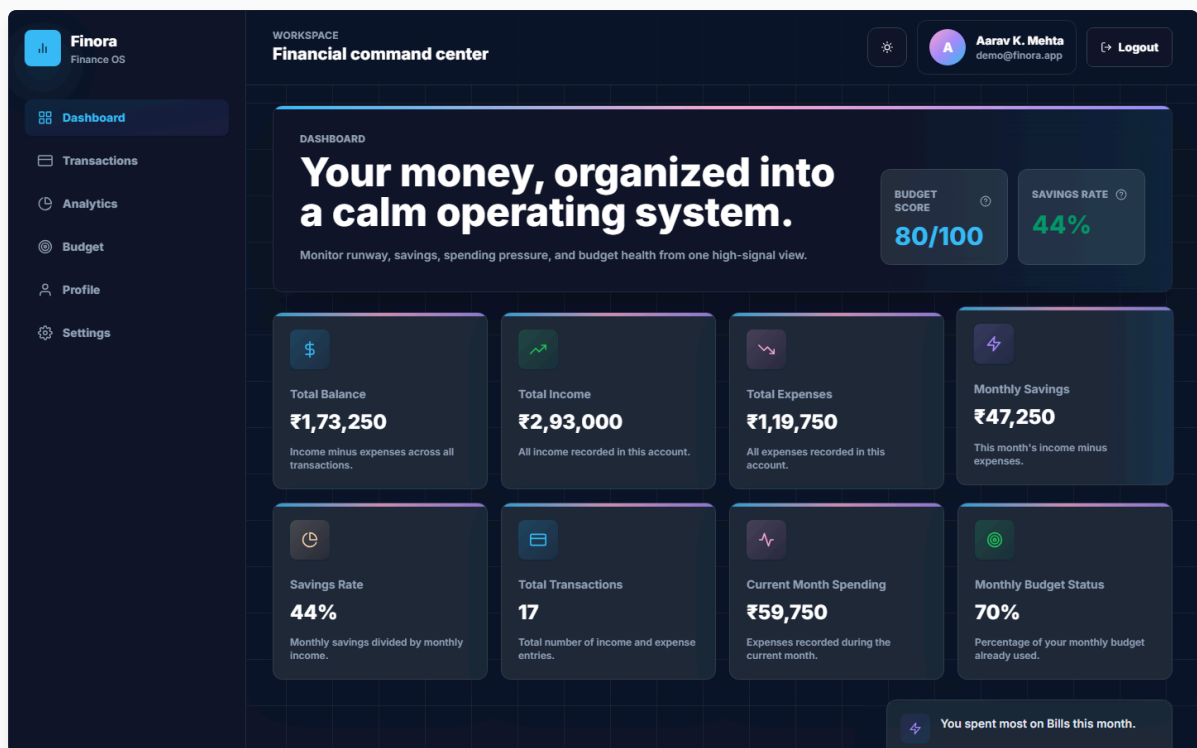
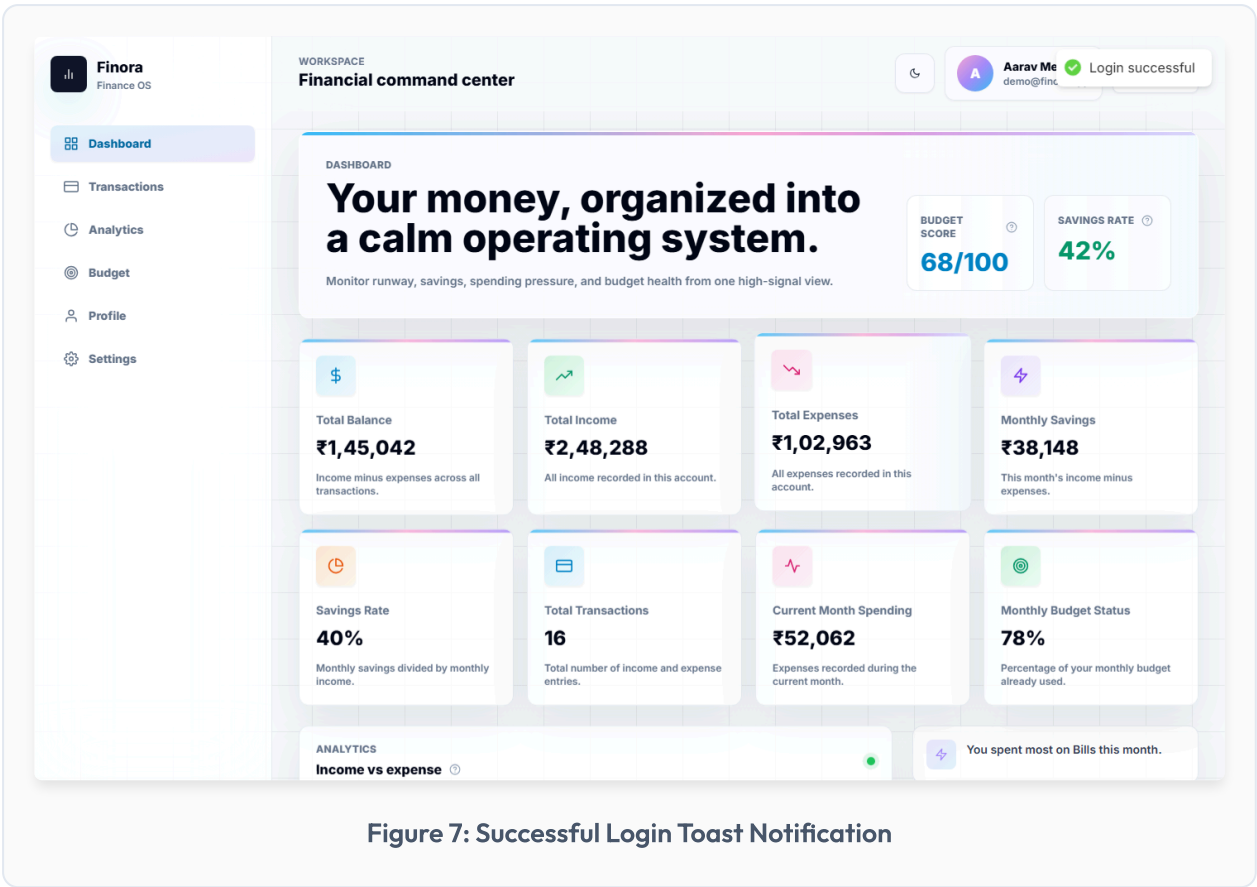
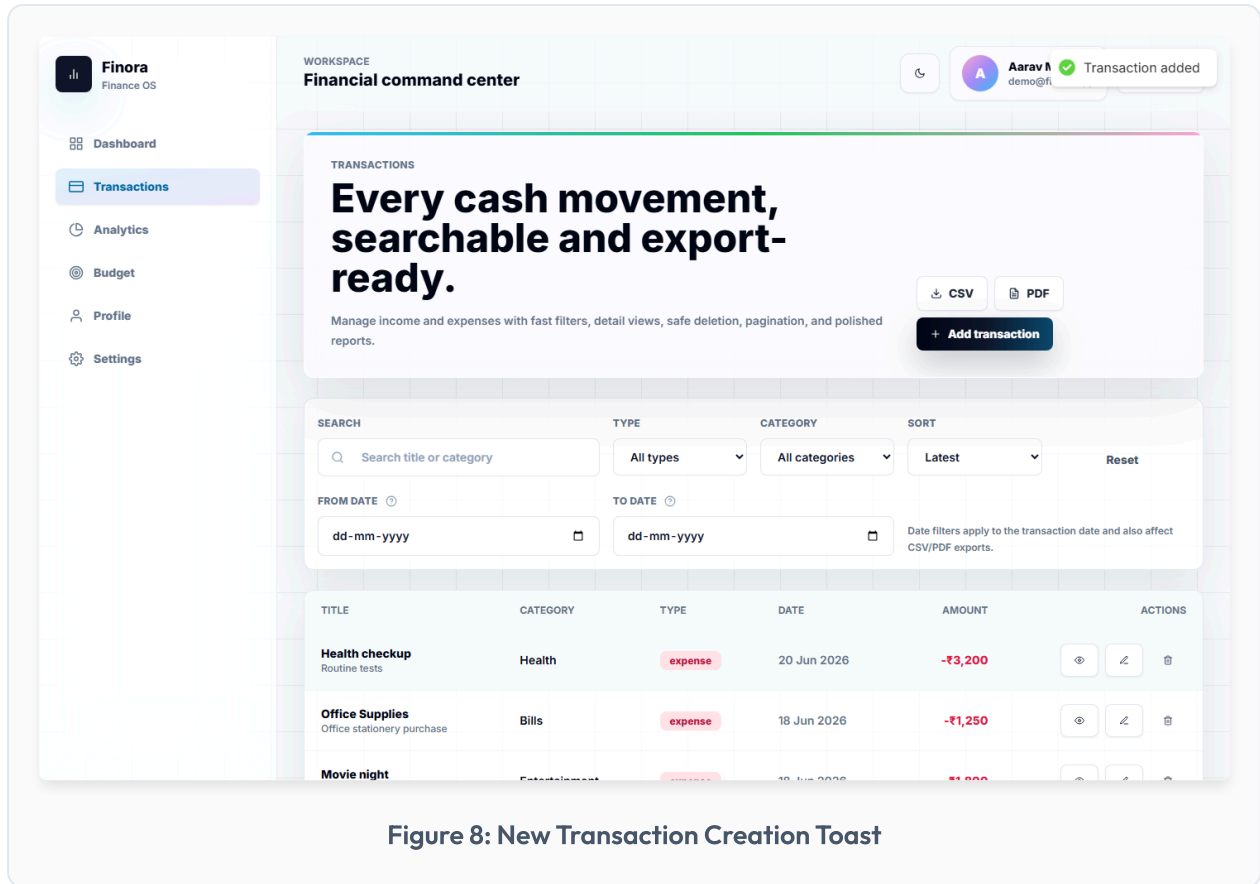


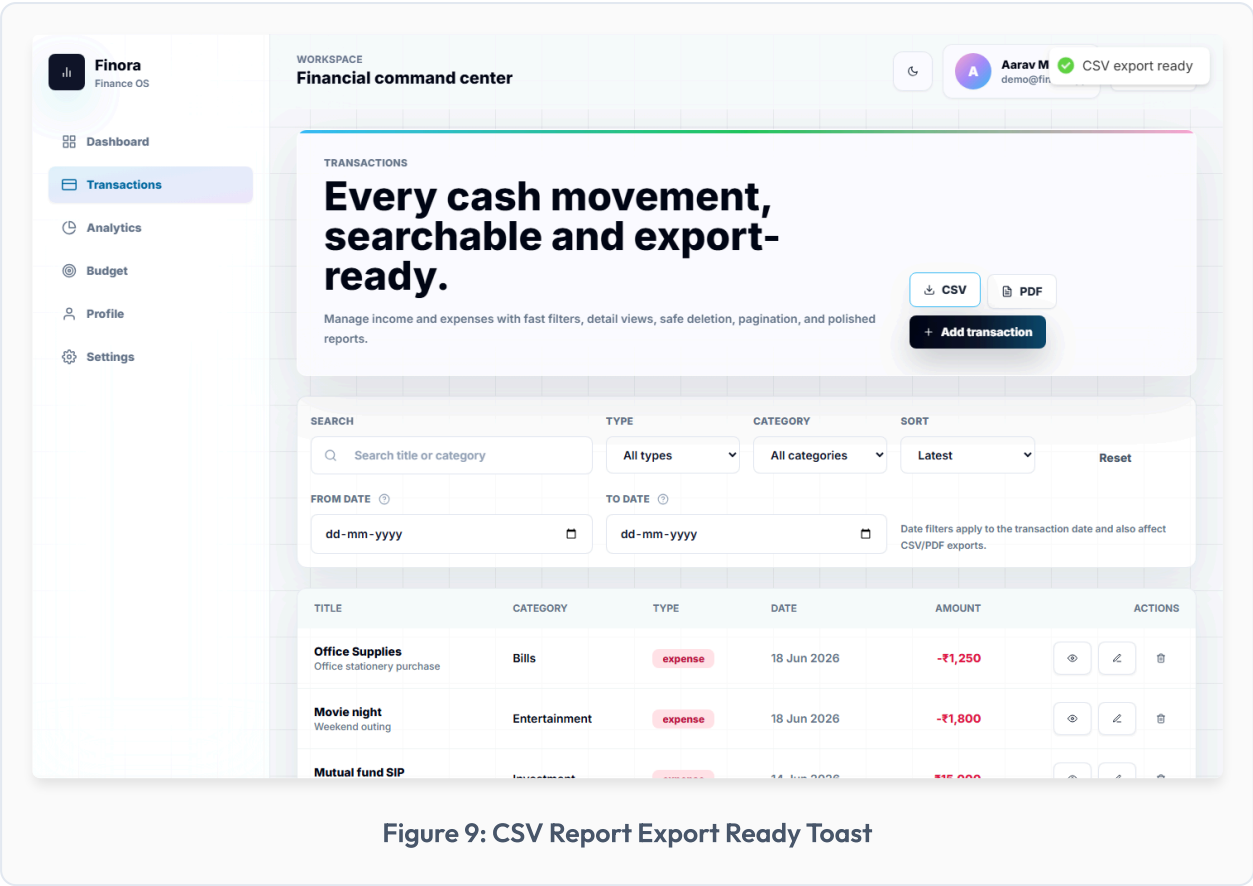
Figure 6: Premium Dark Mode Dashboard

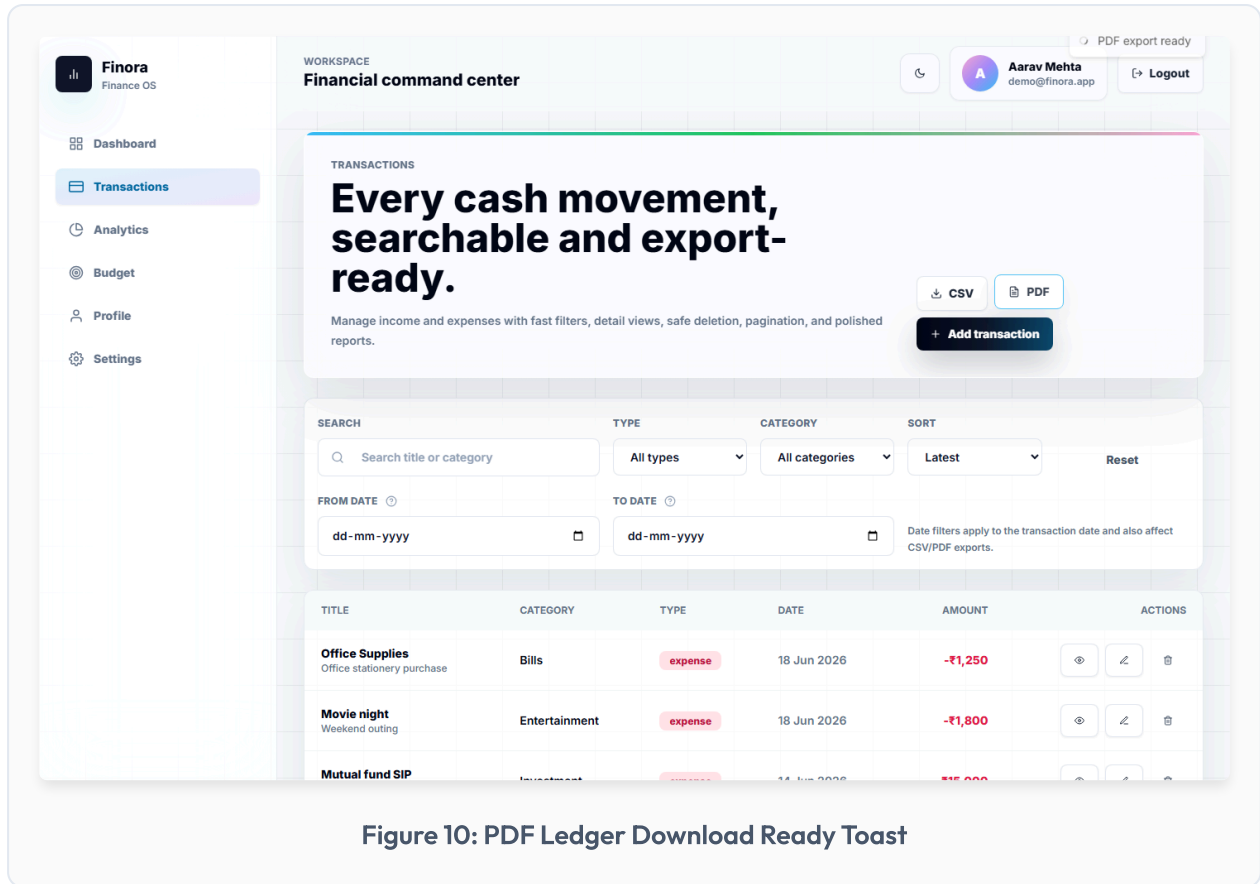
16. Output Verification Images

The following captures document success notifications and verification actions executed in the application flow.









17. Deployment Architecture

Finora uses a production-ready cloud deployment model. The frontend build is deployed to **Vercel** edge routing, providing instant load times for static React bundles. The backend API is hosted on **Render** node containers. The production database relies on **MongoDB Atlas** shared replications. HTTPS and SSL controls secure communication.

18. QA Testing Process

The application has undergone thorough manual and automated end-to-end routing validation. The test matrix documented in `TESTING_REPORT.md` lists all 19 test scenarios, confirming all features operate with 100% pass rates.

19. Challenges Faced

Developing a secure finance dashboard within 8 weeks introduced several technical challenges. Implementing responsive charts that resize automatically during window scaling was resolved by utilizing flexible layouts and responsive chart wrappers. Handling third-party script vulnerabilities was mitigated by setting strict Helmet Content Security Policy rules. Database connection downtime was addressed by creating a safe local demo mode fallback.

20. Future Enhancements

- **Bank Account Sync:** Automatic transaction sync using Plaid APIs.
- **AI Financial Advisory:** LLM models summarizing spending habits and generating advice.
- **Receipt Scan:** Image parser extracting prices from uploaded photos.

21. Conclusion

Finora successfully delivers a premium personal finance tracking workspace that meets all capability standards. The Capstone project fulfills all development constraints and internship submission requirements set by Codtech IT Solutions Private Limited.